

1. Executive Summary

Statur is a self-sovereign, pseudonymous reputation system designed to bring verifiable behavioral trust to decentralized ecosystems. Built for Cardano, Statur enables individuals and organizations to build and share reputation data through a secure hybrid architecture that combines on-chain proofs with flexible off-chain computation.

At its core, Statur allows users to mint a soulbound identity token that anchors their reputation on-chain while keeping detailed reputation data off-chain for scalability. The token is held by the reputation contract and bound to the user's wallet by its public key hash, so it never sits in the user's wallet and cannot be moved to another user. Projects can define metrics and badges, sign updates using cryptographic keys, and contribute to a user's reputation only with explicit authorization. All updates are fully traceable and verifiable through a canonical hashing model, ensuring both transparency and user sovereignty.

Statur's design addresses several long-standing challenges in decentralized identity systems:

- **Trust Establishment:** Users frequently interact under pseudonymous identities. Statur enables trust-building without compromising privacy.
- **Data Sovereignty:** Users control which projects may update their reputation and may revoke access at any time.
- **Verifiability:** All updates are signed, historically preserved, and reconciled on-chain using cryptographic hashes.
- **Scalability:** Heavy or frequently changing data is kept off-chain, while the blockchain stores a snapshot of a user's reputation data as a hash.

Reputation scoring is based on project-defined metrics and weights. Each metric stores a floating-point value, and weighted values are combined using a simple cartesian product to produce a project-level score. While Statur originally explored calculating a global cross-project reputation score, project governance and protocol integrity concerns led to postponing this feature.

This document provides a high-level yet technically accurate description of Statur's purpose, architecture, foundational concepts, and design motivations. It is intended for a mixed audience of grant reviewers, business stakeholders, and technical practitioners.

2. Introduction

2.1 Purpose of This Document

This document outlines the conceptual and architectural foundations of the Statur reputation system. It is written to be accessible to non-engineers while providing the structural clarity needed for technical readers to understand how the system operates and how it will evolve.

The document covers:

- Statur's goals and motivating problems
- Key concepts (users, projects, metrics, badges, scoring model, etc.)
- High-level architecture and design principles
- The hybrid on-chain/off-chain reputation model
- The rationale behind architectural decisions

This document does **not** include API specifications, database schema diagrams, UI/UX wireframes, or smart contract code. These artifacts are delivered separately as part of the broader milestone outputs.

2.2 Background and Motivation

Most decentralized ecosystems rely on pseudonymous identities with public keys rather than traditional user accounts. While this protects individual privacy, it also creates friction: it becomes difficult to assess whether an unknown participant is trustworthy, reliable, or aligned with ecosystem norms.

Existing reputation systems suffer from several limitations:

- Centralized control or opaque scoring
- Limited ability to share reputation across applications
- Lack of user consent in data aggregation
- Inability to verify updates independently
- High on-chain storage costs for dynamic reputation data

Statur addresses these gaps with a self-sovereign, verifiable, flexible reputation model built on:

- Soulbound identity tokens anchoring reputation
- Project-signed metric updates stored in a traceable history
- On-chain hashing to ensure reproducibility and integrity
- Off-chain canonical state to support complex data structures
- User-controlled authorization governing project access

This architecture supports lagon's decentralized storage ecosystem while remaining adaptable for future projects, enterprises, and cross-industry applications.

2.3 Terminology and Definitions (High-Level)

A full glossary appears later, but key concepts include:

User

A pseudonymous participant who mints a soulbound token and decides which projects may contribute to their reputation.

On-Chain Identity (Wallet)

A blockchain-level identity represented by a public key hash.

This identity is associated with the soulbound reputation token and is the anchor for reputation storage and verification.

Project

A business or application that defines metrics, issues badges, and signs updates using a dedicated project key.

Metric

A floating-point value representing some quantitative or qualitative aspect of user behavior.

Metrics may also function as boolean or enumerated flags. Each metric has a project-assigned weight for scoring.

Badge

An achievement-like marker that represents accomplishments or states. Badges are immutable once assigned.

Client

An authenticated API entity representing a project administrator, integration service, or analyst.

Reputation State

A canonical off-chain data structure representing all metrics, badges, and historical updates for a user.

On-Chain Anchor

A hash of the canonical reputation state stored in the smart contract's UTxO, ensuring verifiability.

3. Statur Overview

3.1 System Purpose and Vision

Statur provides a robust, privacy-preserving mechanism for building trust in decentralized networks. It empowers users to curate their reputations while giving projects the means to publish trustworthy behavioral indicators without centralizing control.

The vision of Statur aligns with the objectives outlined in the original Catalyst proposal: to create a reputation model for Cardano that encourages positive behavior, strengthens ecosystem collaboration, and expands toward a chain-agnostic reputation layer.

Core pillars of the system's vision:

- **User Sovereignty:** Users decide which projects may influence their reputation.
- **Transparency:** Updates are signed, immutable, and traceable.
- **Interoperability:** Statur supports heterogeneous metrics across many projects.
- **Verifiability:** Hash-based anchoring ensures that off-chain reputation data is tamper-evident.
- **Scalability:** The hybrid architecture avoids excessive on-chain storage and fees.

Statur introduces a framework for ecosystem-level behavioral insights without compromising the decentralization ethos.

3.2 Design Principles

Statur is built on several foundational principles:

Self-Sovereignty

Reputation belongs to the user. Projects cannot modify data without explicit permission, and users may disconnect from projects at any time.

Minimal Trust Assumptions

The system minimizes trust as much as possible. Cryptographic signatures and canonical hashing make updates independently verifiable.

Scalability Through Hybrid Design

Dynamic reputation data lives off-chain, while the blockchain stores only immutable references. The result is a cost-efficient, high-throughput model.

Auditability

Every update forms part of a linear, signed transition history, allowing users or auditors to reconstruct the complete reputation state.

Project Flexibility

Each project defines its own metrics, weights, and badges. The system allows projects to calculate custom top-level scores using their own algorithms if desired.

3.3 Key Components at a Glance

Soulbound Identity Token

A token held in a contract UTxO that anchors a user's reputation. The token stays inside the reputation contract and is bound to the user's wallet by that wallet's public key hash, which makes it non-transferable: the user never holds it in their wallet and so cannot send it to another user.

Off-Chain Reputation Engine

A canonical state machine that maintains reputation data, updates, and history. Designed to be reproducible and verifiable.

Metrics and Weights

Projects define floating-point metrics and assign weights. A user's per-project score is the weighted average of their active metric values for that project, optionally damped by an exponential decay factor when the membership is inactive. The full formula is given in Section 7.3.

A global cross-project score is intentionally excluded at this stage due abuse concerns and lack of agreed upon use case.

Badges

Immutable, project-defined achievements that augment reputational insight.

Project Signing Keys

Each project uses an ed25519 private key to sign updates to user data, ensuring integrity and authorization.

Backend API

Authenticated endpoints that manage projects, metrics, badges, memberships, and signed updates. (API specification referenced separately.)

On-Chain Hash

A Blake2b-256 hash of the user's complete reputation state, enabling inexpensive verification through smart contracts.

Batcher Process

A scheduled mechanism that publishes aggregated reputation updates to the blockchain efficiently when needed.

4. Core Concepts

This section provides a structured, reader-friendly description of the foundational concepts that make up the Statur ecosystem. These concepts are intentionally described at a high level so both business and technical stakeholders can understand how the system operates without requiring deep blockchain or cryptographic expertise.

4.1 Users

A user is an individual or organization participating in Statur. Users interact with Statur using a blockchain account and mint a soulbound identity token as part of their onboarding. This token uniquely anchors their reputation within the system.

Key characteristics:

- Each user has one reputation identity, anchored by one soulbound token.
- Users control which projects may contribute updates to their reputation.
- Users may leave the system at any time, which spends their reputation UTxO and burns the soulbound token, removing the user's reputation from the active on-chain state. Historical transactions, including all past on-chain reputation hashes, remain permanently on the ledger as a property of the underlying blockchain.
- Users rejoining the system will retain their previous identity and restore the visibility of their previous reputation.

Users do *not* need to actively manage technical details; participation is primarily governed by granting or revoking project access.

4.2 Projects

A project is any entity (business, protocol, application, or community) that wants to measure user behavior, issue badges, or maintain a project-specific reputation model.

Projects:

- Register through the backend and receive a client and API key.
- Generate an ed25519 signing keypair used to authorize all updates.
- Manage their own metrics and badges.
- Decide what behaviors or attributes they consider relevant.

Projects and users form explicit relationships stored in the backend (see project-user relationships in the DB diagram in Appendix A).

4.3 Accounts

An account in Statur is any logical actor that uses the system. Accounts exist at the Statur application level and represent:

- End users who participate in projects and have their reputation tracked
- Project administrators who configure metrics, badges, and manage project settings
- Analyst or reader accounts that query and aggregate public reputation data across one or more projects

Accounts are authenticated entities that:

- Log in to the Statur backend (or integrate via API keys and JWTs)
- Are associated with specific roles and subscription tiers
- May be linked to one or more blockchain identities (wallets) that actually hold the soulbound tokens used for on-chain anchoring

In the implementation, blockchain-facing data (such as the `public_key_hash` of a wallet) is stored separately from these logical accounts. The database `Account` table models on-chain identity for reputation anchoring, while Statur “accounts” are a broader concept used to represent who is interacting with the system and in what capacity.

4.4 Clients

A client represents an authenticated actor in the system, typically:

- A project administrator
- A backend integration
- An analytics consumer

Clients receive:

- An API key
- A tier (which governs rate limits and capabilities)

Clients authenticate using JWTs derived from their API keys.

4.5 Metrics

Metrics are the core building blocks of project-defined reputation.

Properties:

- Represented as floating-point numbers (minimum 0.0, maximum 2 to the power of 63).

- May serve as:
 - Continuous values
 - Boolean indicators (0 or 1)
 - Bitmasks (e.g., representing a user's role and permission set within a project, where flag 1 = Editor, flag 2 = Contributor, flag 4 = Moderator, allowing multiple roles to be combined into a single integer)
 - Enumerations (integer-coded metadata for single choice options)
- Each metric has:
 - A key (immutable identifier)
 - An optional label and unit
 - A weight (0 or above), which influences reputation scoring
 - An optional description for display or documentation

Projects update metric values for connected users using signed requests.

4.6 Badges

Badges are achievement-like markers used to highlight milestones, contributions, or statuses. They are:

- Immutable once assigned
- Associated with a project
- Defined with at minimum:
 - A name
 - An image URI
 - Optional metadata JSON

Once assigned, badges provide qualitative context to a user's reputation.

5. High-Level System Architecture

This section provides a top-level view of Statur's hybrid on-chain/off-chain architecture. The design intentionally separates concerns to achieve:

- Cost-efficiency
- User sovereignty
- Cryptographic verifiability
- Scalability across future chains

5.1 Architectural Overview

Statur is implemented as a hybrid on-chain/off-chain system composed of four core subsystems. Together, they provide scalable reputation computation, cryptographic verification, efficient data access, and deterministic state anchoring on the blockchain.

Blockchain Layer

The blockchain layer contains:

- The soulbound identity token policy, used to anchor a user's on-chain reputation identity.
- The reputation contract, which holds one UTxO per participating wallet and stores the current hash of the user's canonical reputation document.
- The fee contract, which users fund to pay for future batch updates.

Responsibilities:

- Enforce non-transferability of the soulbound token. The token is held by the reputation contract rather than by the user's wallet; the wallet is bound to it through a public key hash recorded in the UTxO datum. Because the user never holds the token, they cannot send it to another wallet, and the validator constrains datum transitions so the bound public key hash cannot be rewritten.
- Maintain the authoritative on-chain hash of each user's reputation.
- Provide deterministic validation rules for batch updates and user-triggered updates.

The blockchain layer stores only minimal, immutable data to ensure low fees and maximum scalability.

Off-Chain Reputation Engine (Backend + REST API)

This subsystem combines the REST API and the backend logic responsible for:

- Managing projects, metrics, badges, and permissions.
- Processing and validating signed metric updates from projects.
- Maintaining the canonical reputation document for each wallet identity.
- Applying deterministic ordering and serialization rules to ensure reproducibility.
- Computing per-project reputation scores using metric values and weights.
- Generating the Blake2b-256 hash that is later published on-chain.
- Exposing authenticated REST endpoints for integrators, project admins, and internal services.

The Off-Chain Reputation Engine is the logical “center” of Statur, coordinating all updates, state transitions, and verification mechanisms.

Database Layer

The relational backend (defined in the DBML schema linked in Appendix A) stores:

- Statur user accounts and API clients
- Project definitions and configuration
- Metric definitions and metric values
- Badge definitions and assignments
- Wallet identities (on-chain representation)
- Project–wallet memberships
- Historical signed transitions
- Derived reputation scores

This layer provides strong consistency, auditability, and efficient lookup of reputation data, while maintaining a strict separation between application-level accounts and blockchain-level identities.

Batcher / Update Service

The batcher is responsible for efficiently synchronizing off-chain reputation changes with the blockchain.

Responsibilities:

- Detecting when a user’s canonical reputation document has changed.
- Recomputing the canonical hash for each updated identity.
- Constructing and submitting a chain of transactions that update multiple users’ reputation UTxOs.
- Funding transactions using user-supplied fee UTxOs stored in the fee contract.
- Ensuring parallel, scalable updates without imposing unnecessary costs on operators.

Users may also initiate manual updates, but the batcher provides the standard, automated mechanism for on-chain synchronization.

5.2 Component Architecture

5.2.1 Blockchain Layer

The blockchain layer includes:

- The soulbound token policy
- The reputation contract (holding user UTxOs)
- The fee contract
- Validation logic ensuring:
 - The identity token remains locked

- Only authorized updates (via batcher) can change the on-chain state
- Users can exit by burning the token

The on-chain data contains only:

- The soulbound token
- The current Blake2b-256 hash of the user's canonical off-chain reputation document

This design keeps on-chain costs predictable.

5.2.2 Off-Chain Reputation Engine

This subsystem maintains the *canonical reputation documents* for each user, containing:

- Metrics and their values
- Badges
- Historical transitions
- Timestamps and signatures

The engine ensures:

- Deterministic canonicalization
- Reproducibility (a user or auditor can recompute the hash independently)
- Permission checks for project updates
- Automatic score calculation per project

The off-chain engine is the heart of the Statur model (an example flow for updating a user metric and how the engine contributes to this is described in:

<https://github.com/lagonAS/iagon-statur-reputation-system/blob/main/docs/diagrams/sequence-metric-account-update.md>).

5.2.3 Backend API Layer

The API layer offers endpoints for:

- Project registration
- Metric and badge creation
- User membership management
- Submitting signed metric updates
- Retrieving user reputation and scores

It performs:

- Authentication
- Signature verification

- Access control checks
- Reputation recomputation and hashing

It is the primary integration point for external applications.

5.2.4 Data Storage Layer

The relational schema

(<https://github.com/lagonAS/iagon-statur-reputation-system/blob/main/docs/diagrams/reputation-database-schema.svg>) supports:

- Users and clients
- Projects
- Metric definitions and metric entries
- Badge definitions and badge assignments
- Project-user relationships
- Historical reputation transitions
- Computed reputation values

This snapshot-oriented design ensures:

- Full traceability
- Efficient querying
- Logical separation between definitions and entries

5.2.5 Sample Backend/Frontend and SDK

The sample Frontend/Backend components include:

- Project administration interfaces
- User reputation viewers
- Flow demonstrations for onboarding and metric updates

The SDK provides:

- Client wrappers for REST endpoints
- Helpers for constructing and signing update payloads
- Utilities for hash verification and canonicalization

6. Smart Contract Architecture

6.1 Design Rationale

The Statur smart contract suite is designed to provide strong integrity guarantees while minimizing on-chain storage and computation costs. Rather than storing full reputation data on chain, only a Blake2b-256 hash of the canonical reputation document is retained on-chain. This approach allows:

- Predictable and low-cost updates.
- Scalable off-chain data structures.
- Fully auditable, cryptographically reproducible verification.

The Cardano eUTxO model is well suited for Statur's update semantics because it allows a stateful contract to maintain a single reputation UTxO per user while supporting parallelized updates and deterministic validation rules.

6.2 Soulbound Token Design

Each user mints a soulbound identity token when entering the reputation contract. The token is held by the reputation contract itself rather than by the user's wallet, and is bound to the user's wallet by the wallet's public key hash, which is stored in the UTxO datum. Statur and external integrators resolve a wallet's reputation by querying the contract for the UTxO whose datum carries that public key hash.

The token has the following characteristics:

- Non-transferable: The user never holds the token in their wallet, so they cannot send it to another wallet. Validator logic prevents the token from being spent to any address other than the reputation contract itself, and constrains datum transitions so the bound public key hash cannot be rewritten to point at a different wallet.
- Burnable on exit: If a user leaves the system, the wallet bound to the token authorizes spending the UTxO in a transaction that burns the token.
- Unique identity anchor: Only one identity token is ever associated with a user's reputation.

This ensures that reputational data cannot be transferred between users and remains tied to the original identity throughout its lifetime.

6.3 Reputation State Machine

The contract maintains the user's reputation as a single UTxO locked by the validator. The UTxO holds the soulbound NFT in its value, with an inline datum that carries three fields:

- **keeper** — the wallet's verification key hash, identifying the wallet bound to this reputation.
- **pointer** — the asset name of the soulbound NFT held in this UTxO. The asset name is derived at mint from the first input's **OutputReference** (transaction id and output index), which guarantees uniqueness without any external registry. Storing the name in the datum is a denormalization that lets the validator identify the bound token cheaply on every spend, without scanning the input's value.
- **info_hash** — the Blake2b-256 hash of the canonical off-chain reputation document.

The link between the soulbound token and the contract UTxO is established at mint: the validator requires the freshly minted NFT to be paid to the reputation contract address. The NFT itself, held by the script, is what binds the UTxO to the user's reputation — it acts as a state-thread token across the lifetime of the identity. On every subsequent spend, the validator enforces that:

1. The same NFT (matched by **pointer**) remains in the same script address.
2. **keeper** and **pointer** are preserved in the new datum.
3. Only **info_hash** may change, and only as part of an authorized batcher or user-triggered update.

This is a proprietary design rather than an implementation of an existing standard. To our knowledge no ratified Cardano CIP defines a soulbound-token pattern. The closest standard is [CIP-68](#) (Datum Metadata Standard), which uses a reference NFT held by a script paired with a user token held in the wallet; Statur deliberately omits the user-side token so that no asset associated with the reputation ever sits in the user's wallet.

This creates a hybrid state machine where off-chain updates form the logical state transitions, and on-chain hashes serve as checkpoints ensuring veracity.

6.4 Update Mechanism

Two mechanisms update the on-chain reputation state:

Batcher Updates

A dedicated batcher:

- Reads updated off-chain reputation documents.
- Recomputes canonical hashes.
- Publishes updates for many users in an efficient manner, while reducing the likelihood of chain congestion. Initially, the batcher spaces out transactions to leave 2-3 free blocks

between batch updates. In the future, as Cardano's scaling efforts become available, transaction chains or other methods may be used.

- Uses user-funded fee UTxOs to minimize project costs.

User-Triggered Updates

Users may trigger updates manually at any time. This is useful during:

- Sensitive reputation changes.
- Project onboarding flows.
- Verifier-driven checks.

6.5 Security Considerations

Key security guarantees include:

- **Replay protection:** Updates must reference the current datum hash.
- **Authorized updates only:** Each metric update is signed by a project's ed25519 key.
- **User sovereignty:** Users may withdraw at any time, which burns the token and prevents future updates.
- **Tamper-evident history:** Off-chain transitions form a sequence of signed updates that can be independently reconstructed.
- **On-chain payload is hash-only:** The on-chain data for each user is a Blake2b-256 hash of the canonical off-chain document. Historical transactions remain on the ledger permanently, but the hashes alone do not reveal the underlying reputation content; reconstructing it requires access to the corresponding off-chain document.

The security model ensures that neither projects nor operators can forge or manipulate reputational data.

7. Reputation Model

7.1 Conceptual Model

The reputation system is built on canonical off-chain documents that include:

- All metric definitions relevant to the user.
- The user's current metric values.
- All badges the user has earned.
- A complete historical transition log.

The canonical representation ensures that any independent verifier can reconstruct the user's reputation and confirm that its hash matches the on-chain anchor over time.

7.2 Hashing and On-Chain Verification

Statur uses:

- A canonical serializer for the document.
- Blake2b-256 hashing for deterministic output.

The verification pipeline:

1. Fetch the full off-chain document.
2. Canonically serialize its contents.
3. Recompute Blake2b-256.
4. Compare with the on-chain hash.

If the hashes match, the reputation is verified.

7.3 Scoring Model

Each project defines a set of metrics, each with a non-negative floating-point weight (w_i) and a non-negative floating-point value (v_i) per user. Both v_i and w_i are constrained to the range $[0, 2^{64} - 1]$. A user's per-project reputation score is the weighted average of the user's active metric values for that project, optionally damped by an exponential decay factor when the user's membership in the project is inactive:

None

$$\text{score} = \left(\sum_i v_i \cdot w_i / \sum_i w_i \right) \cdot \text{decay}$$

If $\sum_i w_i = 0$ (no active metrics with non-zero weight), the score is defined as 0.

Decay term

None

<code>decay = 1</code>	<code>if the membership is active</code>
<code>decay = 1</code>	<code>if score_decay = 0</code>
<code>decay = 1</code>	<code>if days_inactive < 1</code>
<code>decay = exp(-days_inactive / score_decay)</code>	<code>otherwise</code>

`score_decay` is a per-project integer in days (default 30; half-life $\approx 0.693 \cdot \text{score_decay}$).

`days_inactive` is the whole number of days elapsed since the membership was deactivated.

Decay only applies once a full day has elapsed, which avoids penalising same-day deactivations.

Worked example

Suppose a project defines three metrics with the following weights, and a user has the following values:

Metric	Weight w_i	Value v_i	$v_i \cdot w_i$
uptime	3.0	0.95	2.85
read_speed	1.0	0.80	0.80
write_speed	1.0	0.70	0.70

Then $\sum w_i = 5.0$, $\sum v_i \cdot w_i = 4.35$, and the base score is $4.35 / 5.0 = 0.87$.

If the membership is inactive, has been deactivated for 60 days, and the project's `score_decay` is 30, the final score is $0.87 \cdot \exp(-60 / 30) \approx 0.87 \cdot 0.1353 \approx 0.118$.

Custom project scores

Statur does not run project-supplied scoring algorithms; the only score it computes is the weighted average defined above. Projects can still expose a custom top-level score in two ways, both built on the existing metric and weight model:

1. **Designate a single "score" metric.** The project computes its custom score off-Statur using whatever proprietary algorithm it chooses, then submits the result as a signed metric update for a dedicated metric (for example, `custom_score`) with weight 1. Any other metrics the project wants to track but exclude from the score are kept with weight 0. Because $\sum v_i \cdot w_i / \sum w_i$ then collapses to the single weighted metric's value, the per-project score Statur reports equals the project's custom score, with the configured `score_decay` applied as usual.
2. **Tune weights across multiple metrics.** Projects that want a blended score can simply assign non-zero weights to each contributing metric. The per-project score is then the weighted average of those metrics, with weights chosen by the project.

In both cases, Statur stores and surfaces the values through the standard signed-update flow and computes the per-project score using the formula above; the project retains full control over which metrics contribute and with what weight.

Global Cross-Project Scoring

Although originally proposed, a global aggregated score was determined to be:

- Structurally risky (subject to abuse via self-created projects).
- No globally accepted standard (no universal weighting model fits all ecosystems/assessment dimensions).

The feature is suspended pending future research and community input.

7.4 Qualitative and Quantitative Metrics

Projects may introduce:

- Qualitative assessments (via badges or enumeration metrics).
- Quantitative performance metrics.

Example metrics from Igon's storage network are described below:

Metric Name	Description
Read Speed	Average time taken by a node to retrieve stored data.
Write Speed	Average time required to write data to the node.
Download Speed	Download throughput measured during user data retrieval.
Upload Speed	Upload throughput measured during data storage to the node.
Uptime	Percentage of time the node remained online and available.
Pledge Amount	Total ADA pledged by the node operator.
Delegation Amount	Total ADA delegated by users to the node.
Number of Delegators	Number of individual delegators to the node.
Node Size	Total storage capacity available on the node.
Minimum Delegation	Minimum amount of ADA accepted by the node.
Maximum Delegation	Maximum amount of ADA accepted by the node.

Metric Name	Description
Margin Percentage	Percentage of rewards taken as margin by the node operator.

8. Project and User Workflows

8.1 User Lifecycle

1. Joining Statur

- Users submit their soulbound token mint transaction as proof to join the ecosystem.
- A reputation UTxO is created in the reputation contract.
- An initial on-chain hash of their reputation is stored.

2. Connecting to Projects

- User grants authorization to a project.
- Backend creates a project-user membership entry.
- The linked project can now publish signed metric updates.

3. Viewing Reputation

Users can:

- View their metrics.
- Inspect badges.
- Review update history.
- Independently verify the latest on-chain hash.

4. Leaving Statur

- Users submit their soulbound token burn transaction as proof to leave the ecosystem.
- Users spend their reputation UTxO.
- Soulbound token is burned.
- The user's reputation is removed from the active on-chain state and Statur produces no further on-chain reputation updates for that wallet. Past transactions, including previously published reputation hashes, remain on the ledger permanently. Because the on-chain payload is only a Blake2b-256 hash, the historical record does not reveal the underlying reputation content without the corresponding off-chain document.

8.2 Project Lifecycle

1. Project Registration

- A client is created.
- Project administrators generate signing keys.
- Metrics and badges are configured.

2. Updating User Reputation

- Project submits signed metric updates.
- Backend verifies signatures and authorizations.
- Reputation engine recomputes canonical state as needed.

3. Assigning Badges

- Badges are assigned once and are immutable.
- Stored and visible in the reputation backend.

4. Managing Project Assets

Projects may modify:

- Metric definitions.
- Badge definitions.
- Descriptions and labeling metadata.

8.3 Authorization and Signing Workflow

- Projects sign update payloads with ed25519 keys.
- Backend verifies signatures and membership.
- Invalid or unauthorized updates are rejected.
- Transition entries are recorded for audit purposes.

9. Glossary

Badge – A permanent, project-defined achievement associated with user behavior.

Batcher – A service that publishes on-chain reputation updates efficiently.

Canonical Representation – The deterministic ordering and serialization of off-chain reputation data.

Client – An authenticated integration using API keys and JWT.

Historical Transition – A signed update recording how reputation changed over time.

Metric – A weighted and labeled floating-point value representing user behavior.

Project – Any entity defining metrics or badges and signing metric value updates for its

connected users.

Reputation Document – The complete off-chain representation of a user's reputation.

Scoring Model – The combination of metric values and weights used to create a project-level score.

Soulbound Token – A token anchoring reputation on-chain. It is held by the reputation contract rather than the user's wallet, and bound to the wallet by that wallet's public key hash recorded in the UTxO datum. Because the user does not hold the token, they cannot transfer it; it can only be burned when the user exits the system.

UTxO – Cardano's unspent transaction output, used as a state container for reputation data.

10. Appendices

Appendix A – Database Schema Reference

The full relational schema is provided separately as DBML and diagrams. See:

<https://github.com/lagonAS/iagon-statur-reputation-system/blob/main/docs/diagrams/reputation-database-schema.svg>.

Appendix B – API Specification

The full Swagger/OpenAPI specification is provided separately here:

<https://github.com/lagonAS/iagon-statur-reputation-system/blob/main/docs/lagon%20Reputation%20Backend%20API.yaml>. It demonstrates backend endpoints, authentication, and live structures.

Appendix C – Future Evolution

Areas identified for future exploration:

- Governance mechanisms for cross-project scoring.
- Multi-chain identity anchoring.
- Decentralized hosting for reputation backend and raw metric and badge data
- Zero-knowledge proofs for privacy-preserving verification.